Course Outline: Unit Testing en TypeScript

Title: Unit Testing en TypeScript **Learnpack:** + Unit testing en Typescript **Prerequisite:** Introducción al testing **Target Audience:** Beginner developers learning TypeScript testing fundamentals **Total Lessons:** 11 **Format:** Mixed (36% hands-on)

Course Description

This course introduces students to the essential world of unit testing in TypeScript applications. Students will explore the three most popular testing frameworks (Jest, Mocha/Chai, and Vitest), learn to write effective unit tests, and master test planning strategies including Test-Driven Development.

Building on their understanding of general testing principles, students will gain hands-on experience with TypeScript-specific testing challenges and develop the skills to create robust, maintainable test suites. The course emphasizes practical application through real coding exercises and strategic test planning.

By the end of this course, students will confidently choose the right testing framework for their projects, write comprehensive unit tests, and implement TDD workflows in TypeScript applications.

Course Philosophy

This course emphasizes:

- **Framework agnosticism**: Understanding multiple testing tools to make informed choices
- **Test-driven thinking**: Planning tests before writing implementation code
- **TypeScript-first approach**: Leveraging type safety for better test quality
- **Practical application**: Writing tests that actually improve code reliability

---

Course Statistics Summary

Section	Lessons
00 - Welcome	1
01 - Testing Framework Landscape	3
02 - Unit Testing Fundamentals	3
03 - Test Planning and Strategy	2
04 - Assessment and Mastery	2
05 - Outro	1
Total	**11**

Section 00: Welcome

00.0 Welcome to TypeScript Unit Testing 📖

Learning Objectives:

- Understand the importance of unit testing in TypeScript applications
- Identify the core problems this course solves for TypeScript developers

- Preview the testing frameworks and strategies you'll master
- Connect unit testing to overall application reliability and maintainability

Content Outline:

- Why TypeScript needs specialized testing approaches
- Overview of the three major testing frameworks (Jest, Mocha/Chai, Vitest)
- The role of unit testing in the TypeScript development lifecycle
- What you'll be able to accomplish by course completion

Transitions: This welcome sets the stage for exploring the TypeScript testing ecosystem, beginning with a comprehensive survey of available frameworks.

Key Principles:

- **Type safety enhances test quality**: TypeScript's type system helps catch test errors before runtime
- **Framework choice matters**: Different projects benefit from different testing tools
- **Testing is verification**: Unit tests serve as additional consistency checks for application robustness
- **Test-driven mindset**: Planning tests improves both code design and reliability

Examples:

- A TypeScript function that compiles but fails at runtime vs one caught by well-typed tests
- Comparison of the same test written in Jest vs Mocha/Chai vs Vitest
- Real-world scenario where comprehensive unit tests prevented a production bug

---

Section 01: Testing Framework Landscape

01.0 TypeScript Testing Frameworks: Jest, Mocha/Chai, and Vitest 📖

Learning Objectives:

- Compare the strengths and use cases of Jest, Mocha/Chai, and Vitest
- Understand the ecosystem and community around each framework
- Identify which framework best fits different project types
- Recognize the common patterns shared across all three frameworks

Content Outline:

- Jest: The all-in-one testing solution with built-in TypeScript support
- Mocha/Chai: The flexible, modular approach to testing
- Vitest: The modern, Vite-native testing framework
- Performance characteristics and bundle size considerations
- Community support, documentation, and ecosystem maturity
- Decision framework for choosing between frameworks

Transitions: Now that you understand the landscape, we'll move to the practical setup and configuration of each framework.

Key Principles:

- **No universal best framework**: Each tool excels in different scenarios
- **Ecosystem compatibility**: Framework choice affects tooling, plugins, and team workflow
- **Migration considerations**: Understanding similarities helps when switching frameworks
- **Community momentum**: Active development and support influence long-term viability

Examples:

- Large React project scenario → Jest recommendation with rationale
- Lightweight library project → Vitest recommendation with rationale

- Legacy project migration → Mocha/Chai recommendation with rationale

---

01.1 Framework Setup and Basic Configuration 📖

Learning Objectives:

- Configure TypeScript compilation for each testing framework
- Set up proper file organization and naming conventions
- Understand configuration files and their key options
- Establish proper IDE integration for TypeScript testing

Content Outline:

- TypeScript configuration considerations for testing environments
- Package installation and dependency management for each framework
- Configuration files: jest.config.js, mocha.opts, vitest.config.ts
- File naming conventions and directory structure
- IDE setup for test running and debugging
- Common configuration pitfalls and how to avoid them

Transitions: With our frameworks properly configured, we'll write our first tests to see each framework in action.

Key Principles:

- **Configuration consistency**: Proper setup prevents runtime surprises
- **Type checking in tests**: Ensuring tests themselves are type-safe
- **Development workflow**: Configuration should enhance, not hinder, development speed
- **Tool integration**: Good configuration enables IDE features and debugging

Examples:

- Side-by-side configuration comparison for the same TypeScript project
- Common error: TypeScript compilation issues in test files and their solutions
- Development workflow: running tests from IDE vs command line vs CI/CD

01.2 Writing Your First Tests in Each Framework 💻

Challenge Prompt: Create a simple TypeScript utility function that calculates the area of different geometric shapes, then write equivalent unit tests for this function using Jest, Mocha/Chai, and Vitest frameworks to compare their syntax and features.

Solution Code:

```
 // shapes.ts
export interface Shape {
  type: 'circle' | 'rectangle' | 'triangle';
  dimensions: number[];
}

export function calculateArea(shape: Shape): number {
  switch (shape.type) {
    case 'circle':
      return Math.PI * Math.pow(shape.dimensions[0], 2);
    case 'rectangle':
      return shape.dimensions[0] * shape.dimensions[1];
    case 'triangle':
      return 0.5 * shape.dimensions[0] * shape.dimensions[1];
    default:
      throw new Error('Unknown shape type');
```

```
  }
}

// jest.test.ts
import { calculateArea, Shape } from './shapes';

describe('calculateArea with Jest', () => {
  test('calculates circle area correctly', () => {
    const circle: Shape = { type: 'circle', dimensions: [5] };
    expect(calculateArea(circle)).toBeCloseTo(78.54, 2);
  });
});

// mocha.test.ts
import { expect } from 'chai';
import { calculateArea, Shape } from './shapes';

describe('calculateArea with Mocha/Chai', () => {
  it('calculates rectangle area correctly', () => {
    const rectangle: Shape = { type: 'rectangle', dimensions: [4, 6] };
    expect(calculateArea(rectangle)).to.equal(24);
  });
});

// vitest.test.ts
import { describe, it, expect } from 'vitest';
import { calculateArea, Shape } from './shapes';

describe('calculateArea with Vitest', () => {
  it('calculates triangle area correctly', () => {
    const triangle: Shape = { type: 'triangle', dimensions: [8, 10] };
    expect(calculateArea(triangle)).toBe(40);
  });
});
```

Challenge Code:

```
 // shapes.ts
export interface Shape {
  // TODO: Define the Shape interface
}

export function calculateArea(shape: Shape): number {
  // TODO: Implement area calculation logic
}

// jest.test.ts
import { calculateArea, Shape } from './shapes';

describe('calculateArea with Jest', () => {
  test('calculates circle area correctly', () => {
    // TODO: Create circle shape and test area calculation
  });
});

// mocha.test.ts
import { expect } from 'chai';
import { calculateArea, Shape } from './shapes';
```

```
describe('calculateArea with Mocha/Chai', () => {
  it('calculates rectangle area correctly', () => {
    // TODO: Create rectangle shape and test area calculation
  });
});

// vitest.test.ts
import { describe, it, expect } from 'vitest';
import { calculateArea, Shape } from './shapes';

describe('calculateArea with Vitest', () => {
  it('calculates triangle area correctly', () => {
    // TODO: Create triangle shape and test area calculation
  });
});
```

Section 02: Unit Testing Fundamentals

02.0 Understanding Unit Tests and Test Anatomy 📖

Learning Objectives:

- Define what constitutes a proper unit test in TypeScript applications
- Understand the anatomy of well-structured test cases
- Identify the difference between unit tests and other testing types
- Recognize the key components of effective test organization

Content Outline:

- What makes a test a "unit" test vs integration or functional test
- The AAA pattern: Arrange, Act, Assert in TypeScript context
- Test naming conventions and descriptive test cases
- Test isolation and independence principles
- Mocking and stubbing concepts for unit testing
- Anti-pattern: Testing multiple units in a single test (weaving anti-pattern into concept)

Transitions: Understanding test fundamentals prepares us to tackle TypeScript-specific challenges in our testing approach.

Key Principles:

- **Single responsibility**: Each test should verify one specific behavior
- **Independence**: Tests should not depend on other tests or external state
- **Readability**: Test code should clearly communicate what is being tested
- **Repeatability**: Tests should produce consistent results across runs

Examples:

- Good unit test: Testing a pure TypeScript function with clear inputs/outputs
- Bad unit test: Testing multiple functions and external dependencies together
- Anti-pattern example: Tests that must run in a specific order to pass

---

02.1 TypeScript-Specific Testing Considerations 📖

Learning Objectives:

- Leverage TypeScript's type system for better test quality
- Handle generic functions and complex types in tests

- Use type assertions and type guards in test scenarios
- Understand compilation and runtime differences in testing context

Content Outline:

- Testing generic functions and maintaining type safety
- Handling union types and type guards in test cases
- Testing async/await functions and Promise-based code
- Type assertions in tests: when and how to use them safely
- Testing TypeScript enums, interfaces, and custom types
- Anti-pattern: Ignoring TypeScript errors in tests with 'any' types (weaving anti-pattern)

Transitions: With TypeScript-specific knowledge in hand, we'll apply these concepts to build comprehensive unit tests.

Key Principles:

- **Type safety in tests**: Tests should be as type-safe as production code
- **Generic testing**: Properly testing functions that work with multiple types
- **Runtime vs compile-time**: Understanding what TypeScript checks vs what tests verify
- **Type-driven test cases**: Using TypeScript types to identify edge cases

Examples:

- Testing a generic utility function with different type parameters
- Proper testing of union type handling in a TypeScript function
- Anti-pattern example: Using 'any' type to bypass TypeScript checking in tests

---

02.2 Building Comprehensive Unit Tests 💻

Challenge Prompt: Build a TypeScript user validation system with comprehensive unit tests that cover all edge cases, including valid inputs, invalid data types, boundary conditions, and error scenarios using proper TypeScript typing throughout.

Solution Code:

```
 // userValidator.ts
export interface User {
  id: number;
  email: string;
  age: number;
  roles: ('admin' | 'user' | 'guest')[];
}

export class ValidationError extends Error {
  constructor(public field: string, message: string) {
    super(`${field}: ${message}`);
    this.name = 'ValidationError';
  }
}

export function validateUser(userData: any): User {
  if (typeof userData.id !== 'number' || userData.id <= 0) {
    throw new ValidationError('id', 'must be a positive number');
  }

  const emailRegex = /^[^\s@]+@[^\s@]+\.[^\s@]+$/;
  if (typeof userData.email !== 'string' || !emailRegex.test(userData.email)) {
    throw new ValidationError('email', 'must be a valid email address');
  }
```

```
  if (typeof userData.age !== 'number' || userData.age < 13 || userData.age > 120) {
    throw new ValidationError('age', 'must be between 13 and 120');
  }

  if (!Array.isArray(userData.roles) || userData.roles.length === 0) {
    throw new ValidationError('roles', 'must be a non-empty array');
  }

  return userData as User;
}

// userValidator.test.ts
import { validateUser, ValidationError, User } from './userValidator';

describe('validateUser', () => {
  it('validates correct user data', () => {
    const validUser = {
      id: 1,
      email: 'test@example.com',
      age: 25,
      roles: ['user'] as const
    };

    const result = validateUser(validUser);
    expect(result).toEqual(validUser);
  });

  it('throws ValidationError for invalid email', () => {
    const invalidUser = { id: 1, email: 'invalid-email', age: 25, roles: ['user'] };

    expect(() => validateUser(invalidUser)).toThrow(ValidationError);
    expect(() => validateUser(invalidUser)).toThrow('email: must be a valid email address');
  });

  it('throws ValidationError for boundary age values', () => {
    const tooYoung = { id: 1, email: 'test@example.com', age: 12, roles: ['user'] };
    const tooOld = { id: 1, email: 'test@example.com', age: 121, roles: ['user'] };

    expect(() => validateUser(tooYoung)).toThrow('age: must be between 13 and 120');
    expect(() => validateUser(tooOld)).toThrow('age: must be between 13 and 120');
  });
});
```

Challenge Code:

```
 // userValidator.ts
export interface User {
  // TODO: Define User interface with id, email, age, and roles
}

export class ValidationError extends Error {
  // TODO: Implement custom error class for validation failures
}

export function validateUser(userData: any): User {
  // TODO: Implement user validation logic
  // - Validate id is positive number
  // - Validate email format
  // - Validate age is between 13-120
```

```
  // - Validate roles is non-empty array
}

// userValidator.test.ts
import { validateUser, ValidationError, User } from './userValidator';

describe('validateUser', () => {
  it('validates correct user data', () => {
    // TODO: Test with valid user data
  });

  it('throws ValidationError for invalid email', () => {
    // TODO: Test email validation
  });

  it('throws ValidationError for boundary age values', () => {
    // TODO: Test age boundary conditions
  });

  // TODO: Add more test cases for edge cases
});
```

Section 03: Test Planning and Strategy

03.0 Planning Unit Tests: Coverage, Edge Cases, and TDD 📖

Learning Objectives:

- Develop systematic approaches to identifying test cases
- Understand code coverage metrics and their limitations
- Apply Test-Driven Development principles to TypeScript projects
- Create comprehensive test plans for complex TypeScript functions

Content Outline:

- Identifying representative test cases and edge cases systematically
- Understanding code coverage: line, branch, and function coverage
- The TDD cycle: Red, Green, Refactor in TypeScript context
- Planning tests before implementation to drive better design
- Boundary testing and error condition identification
- Anti-pattern: Over-testing implementation details instead of behavior (weaving anti-pattern)
- Anti-pattern: Testing only the expected result, ignoring error cases (weaving anti-pattern)

Transitions: With a strategic approach to test planning, we'll implement TDD in practice to see these principles in action.

Key Principles:

- **Behavior over implementation**: Test what the function does, not how it does it
- **Edge case identification**: Systematic approach to finding boundary conditions
- **Test-driven design**: Using tests to drive better function interfaces
- **Coverage mindfulness**: Understanding what coverage metrics tell us and what they don't

Examples:

- Systematic test case identification for a TypeScript utility function
- TDD example: Writing tests first to define a function's behavior
- Anti-pattern example: Tests that break when internal implementation changes

---

03.1 Test-Driven Development Implementation 💻

Challenge Prompt: Implement a TypeScript password strength validator using pure TDD methodology - write failing tests first that define the requirements, then implement the minimum code to make each test pass.

Solution Code:

```
 // passwordValidator.test.ts (Written FIRST)
import { validatePasswordStrength, PasswordStrength } from './passwordValidator';

describe('Password Strength Validator (TDD)', () => {
  it('returns WEAK for passwords under 8 characters', () => {
    expect(validatePasswordStrength('short')).toBe(PasswordStrength.WEAK);
  });

  it('returns MEDIUM for passwords 8+ chars with letters only', () => {
    expect(validatePasswordStrength('longpassword')).toBe(PasswordStrength.MEDIUM);
  });

  it('returns STRONG for passwords with letters, numbers, and symbols', () => {
    expect(validatePasswordStrength('MyPass123!')).toBe(PasswordStrength.STRONG);
  });

  it('throws error for empty password', () => {
    expect(() => validatePasswordStrength('')).toThrow('Password cannot be empty');
  });
});

// passwordValidator.ts (Written AFTER tests)
export enum PasswordStrength {
  WEAK = 'weak',
  MEDIUM = 'medium',
  STRONG = 'strong'
}

export function validatePasswordStrength(password: string): PasswordStrength {
  if (password.length === 0) {
    throw new Error('Password cannot be empty');
  }

  if (password.length < 8) {
    return PasswordStrength.WEAK;
  }

  const hasNumbers = /\d/.test(password);
  const hasSymbols = /[!@#$%^&*(),.?":{}|<>]/.test(password);

  if (hasNumbers && hasSymbols) {
    return PasswordStrength.STRONG;
  }

  return PasswordStrength.MEDIUM;
}
```

Challenge Code:

```
 // passwordValidator.test.ts (Write these tests FIRST)
import { validatePasswordStrength, PasswordStrength } from './passwordValidator';
```

```
describe('Password Strength Validator (TDD)', () => {
  it('returns WEAK for passwords under 8 characters', () => {
    // TODO: Write test that will initially FAIL
  });

  it('returns MEDIUM for passwords 8+ chars with letters only', () => {
    // TODO: Write test for medium strength requirement
  });

  it('returns STRONG for passwords with letters, numbers, and symbols', () => {
    // TODO: Write test for strong password requirement
  });

  it('throws error for empty password', () => {
    // TODO: Write test for error condition
  });
});

// passwordValidator.ts (Implement AFTER writing tests)
export enum PasswordStrength {
  // TODO: Define strength levels
}

export function validatePasswordStrength(password: string): PasswordStrength {
  // TODO: Implement logic to make tests pass
  // Start with minimal implementation, then refactor
}
```

Section 04: Assessment and Mastery

04.0 Testing Best Practices and Anti-Patterns 📖

Learning Objectives:

- Identify and avoid common testing anti-patterns in TypeScript
- Apply testing best practices for maintainable test suites
- Understand when to use different types of test doubles (mocks, stubs, spies)
- Establish effective testing workflows for TypeScript projects

Content Outline:

- Best practices: Error testing and handling failure scenarios properly
- Best practices: Making tests readable and maintainable over time
- Test organization and suite structure for large TypeScript projects
- When and how to use mocking effectively in TypeScript
- Anti-pattern deep dive: Over-generation of tests vs focused testing (weaving anti-pattern)
- Anti-pattern deep dive: Testing only happy paths vs comprehensive error testing (weaving anti-pattern)
- Code review practices for TypeScript tests

Transitions: These practices prepare you for the final assessment where you'll demonstrate mastery of TypeScript unit testing.

Key Principles:

- **Test quality over quantity**: Better to have fewer high-quality tests than many brittle tests
- **Error scenarios matter**: Testing failure cases is as important as testing success
- **Maintainability**: Tests should evolve with the codebase without breaking frequently

- **Clear communication**: Tests serve as documentation of expected behavior

Examples:

- Good practice: Comprehensive error testing for a TypeScript API function
- Anti-pattern example: Generating tests for every single method without considering value
- Best practice: Test organization that scales with project growth

---

04.1 TypeScript Testing Mastery Assessment 🧠

Learning Objectives:

- Demonstrate understanding of TypeScript testing framework differences
- Apply unit testing principles to complex scenarios
- Identify appropriate testing strategies for different TypeScript code patterns
- Evaluate test quality and coverage decisions

Question Format: Multiple choice

Questions:

1. When testing a TypeScript generic function `function processData<T>(items: T[]): T[]`, which approach best maintains type safety in your tests? a) Use `any` type to avoid TypeScript complications b) Test with specific types like `string[]` and `number[]` separately c) Only test with one type since generics work the same way d) Skip testing generic functions since TypeScript handles the types

2. In Test-Driven Development with TypeScript, what should you do when your test fails to compile due to TypeScript errors? a) Add `@ts-ignore` comments to bypass the errors b) Fix the TypeScript errors first, then run the test to see it fail c) Skip TDD and write the implementation first d) Use JavaScript instead of TypeScript for tests

3. Which testing scenario represents a proper unit test for a TypeScript function? a) Testing a function that calls an external API and validates the response b) Testing a pure function that calculates tax based on input parameters c) Testing the entire user login flow from UI to database d) Testing multiple functions together to verify they integrate correctly

4. What is the main advantage of Vitest over Jest for TypeScript projects? a) Vitest has better TypeScript type checking b) Vitest is faster and has native ES modules support c) Vitest has more testing features than Jest d) Vitest works better with React components

5. When testing TypeScript error conditions, what is the best practice? a) Only test the happy path since TypeScript prevents most errors b) Test both expected errors (thrown intentionally) and edge cases c) Use try-catch blocks in your implementation instead of testing errors d) Focus only on TypeScript compilation errors, not runtime errors

6. Which represents the anti-pattern of "over-generation of tests"? a) Writing tests for every public method regardless of complexity b) Writing multiple test cases for complex business logic c) Testing both success and failure scenarios d) Writing tests before implementing the code (TDD)

7. In TypeScript testing, when should you use type assertions (`as` keyword) in your tests? a) Frequently, to make tests easier to write b) Sparingly, only when you need to test specific type scenarios c) Never, type assertions make tests unreliable d) Always, to ensure TypeScript doesn't interfere with test logic

Evaluation Criteria:

- **7 correct**: Complete mastery - ready for advanced testing topics
- **5-6 correct**: Good understanding - review framework differences and anti-patterns
- **3-4 correct**: Basic grasp - revisit TDD principles and TypeScript-specific testing
- **0-2 correct**: Needs significant review of core concepts

Score Interpretation: This assessment validates your understanding of TypeScript unit testing fundamentals, framework selection, and best practices. Strong performance indicates readiness to implement comprehensive testing strategies in real TypeScript

projects.

Section 05: Outro

05.0 Your Testing Journey Forward

Content Outline:

- Course recap: You've mastered three major TypeScript testing frameworks and understand when to use each one
- What you've accomplished: From basic test setup to comprehensive TDD implementation
- Connection to bigger picture: How unit testing fits into the broader software quality and development lifecycle
- Next steps: Integration testing, end-to-end testing, and advanced TypeScript testing patterns
- Resources for continued learning: Official documentation, testing communities, and advanced courses
- Encouragement: You now have the foundation to write reliable, maintainable TypeScript applications with confidence

Note: This is conclusion only - no theory, exercises, or assessment.
